



UNIVERSIDAD PERUANA
CAYETANO HEREDIA

MYVOICE: Reconocimiento de comandos simples en dispositivos de voz asistida mediante señales EMG

Código Fuente

Julio 2026

CÓDIGO PYTHON.

```
import streamlit as st
import numpy as np
import pandas as pd
import time
from collections import deque
import emg_inference as ei

st.set_page_config(
    page_title="Clasificador EMG",
    page_icon="🧠",
    layout="wide"
)

# — Personas

PERSONAS = {
    "Persona A — Señal estable": {
        "snr": 12.0, "baseline_drift": 0.02, "burst_amplitude": 3.0,
        "freq_shift": 0.0, "desc": "Buena colocación de electrodos, señal limpia."
    },
    "Persona B — Señal ruidosa": {
        "snr": 4.0, "baseline_drift": 0.15, "burst_amplitude": 2.5,
        "freq_shift": 0.0, "desc": "Electrodos con contacto subóptimo, ruido muscular aumentado."
    },
    "Persona C — Señal con deriva": {
        "snr": 8.0, "baseline_drift": 0.40, "burst_amplitude": 3.5,
        "freq_shift": -0.3, "desc": "Movimiento del cable causando deriva lenta de línea base."
    },
    "Persona D — Señal fatigada": {
        "snr": 6.0, "baseline_drift": 0.08, "burst_amplitude": 1.8,
        "freq_shift": 0.5, "desc": "Fatiga muscular progresiva — amplitud reducida y frecuencia desplazada."
    },
}

# — State init

if "buffer" not in st.session_state:
```

```

    st.session_state.buffer = {ch: deque(maxlen=1000) for ch in
range(4)}
if "history" not in st.session_state:
    st.session_state.history = [] # [(word, prob, persona, time)]
if "persona" not in st.session_state:
    st.session_state.persona = "Persona A - Señal estable"
if "connected" not in st.session_state:
    st.session_state.connected = False
if "prediction_count" not in st.session_state:
    st.session_state.prediction_count = 0

# — Helpers

def generate_emg_window(persona_key, label, n_samples=1000, fs=1000):
    """
    Genera una ventana EMG realista de 4 canales con características
    espectrales diferenciadas por clase:
        - BASAL: solo ruido muscular de baja amplitud
        - NO: burst de media frecuencia (~30-80 Hz)
        - SI: burst de alta frecuencia (~80-200 Hz)
    """
    cfg = PERSONAS[persona_key]
    rng = np.random.RandomState(int(time.time()) * 1e6) % (2**31))

    t = np.arange(n_samples) / fs
    baseline = cfg["baseline_drift"] * np.sin(2 * np.pi * 0.1 * t +
rng.randn()*0.5)
    baseline = baseline[:, None] # (1000, 1)

    emg = np.zeros((n_samples, 4))
    for ch in range(4):
        noise = rng.randn(n_samples) / cfg["snr"] * 0.5
        if label == "BASAL":
            sig = 0.15 * rng.randn(n_samples) + 0.02 *
np.sin(2*np.pi*50*t + rng.rand())
        else:
            # spectral burst
            f_shift = cfg["freq_shift"]
            if label == "NO":
                f0, bw = 55 + f_shift*20, 25
            else: # SI
                f0, bw = 140 + f_shift*40, 60
            phi = rng.uniform(0, 2*np.pi)

```

```

        carrier = np.sin(2*np.pi*f0*t + phi)
        envelope = np.exp(-((t - 0.5)**2) / (2*0.08**2)) #
gaussian burst
        sig = cfg["burst_amplitude"] * envelope * carrier
        sig += 0.05 * rng.randn(n_samples) # residual noise
        emg[:, ch] = sig + noise

    emg += baseline * rng.uniform(0.5, 1.5, (1, 4))
    return emg # (1000, 4)

# — UI

st.title("🧠 Clasificación de palabras mediante señales EMG")
st.caption("Ingeniería Biomédica – BITalino + Edge Impulse + TFLite
Micro")

# — Sidebar

with st.sidebar:
    st.header("⚙️ Configuración")

    prev = st.session_state.persona
    st.session_state.persona = st.selectbox(
        "Seleccionar persona",
        list(PERSONAS.keys()),
        index=list(PERSONAS.keys()).index(st.session_state.persona),
    )

    st.caption(PERSONAS[st.session_state.persona]["desc"])

    col1, col2 = st.columns(2)
    with col1:
        if st.button("🔌 Conectar BITalino", use_container_width=True):
            st.session_state.connected = True
    with col2:
        if st.button("🔌 Desconectar", use_container_width=True):
            st.session_state.connected = False

    st.divider()
    st.subheader("📊 Estadísticas")
    total = len(st.session_state.history)

```

```

st.metric("Predicciones totales", total)
if total > 0:
    dfh = pd.DataFrame(st.session_state.history,
columns=["palabra", "conf", "persona", "ts"])
    counts = dfh["palabra"].value_counts()
    for w in ["BASAL", "NO", "SI"]:
        st.metric(w, counts.get(w, 0))

if st.button("🗑 Limpiar historial", use_container_width=True):
    st.session_state.history = []
    st.rerun()

st.divider()
st.caption("Proyecto EMG – Reconocimiento de palabras")
st.caption(f"v2.0 – {time.strftime('%d/%m/%Y')}")

# — Pestañas principales


---


tab_overview, tab_signals, tab_pipeline, tab_about = st.tabs([
    "📋 Vista General", "📈 Señales EMG", "⚡ Pipeline de Inferencia",
    "📖 Acerca del Proyecto"
])

# — TAB 1: Vista General


---


with tab_overview:
    col_left, col_right = st.columns([3, 2])

    with col_left:
        st.subheader("Sistema de reconocimiento de habla silenciosa")
        st.markdown("""
        Este sistema clasifica intentos de pronunciación de
palabras ("SI", "NO", "BASAL")
        a partir de señales electromiográficas (EMG) de superficie en
4 músculos
        suprahioideos involucrados en la articulación del habla.

        ### Pipeline
        1. Adquisición – BITalino a 1000 Hz, 4 canales
        2. Preprocesamiento – Ventaneo de 1000 muestras (1 s)
        3. DSP – FFT espectral de 64 puntos → 148 features
        4. Normalización – StandardScaler (148 medias, 148 escalas)

```

```

5. **Inferencia** – Red Fully-Connected cuantizada int8
(148→120→30→10→3)

6. **Post-procesamiento** – Softmax + umbral de confianza  $\geq 60\%$ 
    """

    con = st.container(border=True)
    con.markdown("**👤 Estado del dispositivo**")
    if st.session_state.connected:
        con.success(f"✅ BITalino conectado – Persona activa:
**{st.session_state.persona}**")
    else:
        con.info("🛑 BITalino desconectado. Conecta desde la barra
lateral.")

    with col_right:
        st.subheader("📊 Canales musculares")
        st.markdown("""<div style="background:#f0f2f6;
border-radius:10px; padding:1rem; text-align:center;
font-size:0.9rem;">
<b>👉 Músculos suprahioides</b><br>
• Estilohioideo D/I<br>
• Digástrico anterior D/I<br>
• Milohioideo<br>
• Genihioideo
</div>""", unsafe_allow_html=True)
        st.markdown("""
| Canal | Músculo |
|-----|-----|
| A1 | Estilohioideo derecho |
| A2 | Estilohioideo izquierdo |
| A3 | Digástrico anterior derecho |
| A4 | Digástrico anterior izquierdo |
""")

    with st.expander("🧠 ¿Qué mide la EMG?", expanded=False):
        st.markdown("""
        La electromiografía (EMG) mide la **actividad eléctrica
generada por
        las fibras musculares** durante la contracción. En el habla
silenciosa,
        los músculos suprahioides se activan sutilmente al
**intentar** articular
        una palabra, incluso sin emitir sonido.
    """)

```

- ****BASAL****: reposo, solo ruido de fondo muscular
- ****NO****: activación media, patrón espectral ~30-80 Hz
- ****SI****: activación fuerte, patrón espectral ~80-200 Hz

""")

— TAB 2: Señales EMG

```
with tab_signals:
    st.subheader("📈 Visualización de señales EMG en tiempo real")

    row1 = st.columns(2)
    with row1[0]:
        label_to_show = st.radio(
            "Clase a simular",
            ["BASAL", "NO", "SI"],
            horizontal=True,
            key="signal_class"
        )
    with row1[1]:
        n_windows = st.slider("Ventanas a generar", 1, 10, 3)

    if st.button("▶ Generar señales", type="primary",
use_container_width=True):
        with st.spinner("Generando señales EMG sintéticas..."):
            time.sleep(0.3)

            fig_data = []
            all_windows = []
            for w in range(n_windows):
                win = generate_emg_window(st.session_state.persona,
label_to_show)
                all_windows.append(win)
                for ch in range(4):
                    # staggered offset per window for clarity
                    offset = w * 0.5
                    for s in range(len(win)):
                        fig_data.append({
                            "ventana": w + 1,
                            "canal": f"CH{ch+1}",
                            "tiempo (ms)": s,
                            "amplitud (mV)": win[s, ch] + offset,
                        })
```

```

df_plot = pd.DataFrame(fig_data)

import plotly.express as px
fig = px.line(
    df_plot, x="tiempo (ms)", y="amplitud (mV)",
    color="canal", facet_row="ventana",
    title=f"Señal EMG - {label_to_show} -
{st.session_state.persona}",
    height=250 * n_windows + 100,
    color_discrete_map={
        "CH1": "#1f77b4", "CH2": "#ff7f0e",
        "CH3": "#2ca02c", "CH4": "#d62728"
    },
)
fig.update_traces(line=dict(width=1))
fig.update_layout(showlegend=True)
fig.for_each_annotation(lambda a: a.update(text=f"Ventana
{int(a.text.split('=')[1])}"))
st.plotly_chart(fig, use_container_width=True)

# Stats
stacked = np.vstack(all_windows)
col_signal = st.columns(4)
for i, ch in enumerate(["CH1", "CH2", "CH3", "CH4"]):
    col_signal[i].metric(
        f"{ch} - RMS (mV)",
        f"{np.sqrt(np.mean(stacked[:, i]**2)):.3f}"
    )
else:
    st.info("Presiona  Generar señales** para visualizar.")

with st.expander("📖 Interpretación de la señal EMG",
expanded=False):
    st.markdown("""
- Línea base plana (~0 mV): reposo muscular (BASAL)
- Bursts de actividad: contracción muscular al articular
"NO" o "SI"
- Amplitud típica: 0.1-5 mV (depende del músculo,
electrodos, y fatiga)
- Frecuencia: el espectro EMG útil está entre 20-450 Hz
- Persona A (señal estable) muestra patrones limpios;
Persona D (fatiga)
tiene amplitud reducida y espectro desplazado

```



```

        """)

# — TAB 3: Pipeline de Inferencia


---


with tab_pipeline:
    st.subheader("⚡ Pipeline completo: señal → features → clasificación")

    # — Subida de archivos reales
    

---


    st.markdown("### 📁 Datos reales (BITalino TXT)")
    uploaded_file = st.file_uploader(
        "Sube un archivo TXT/CSV de BITalino (4 canales, ≥1000 muestras)",
        type=["txt", "csv"],
        key="emg_upload"
    )

    if uploaded_file is not None:
        with st.status("Procesando archivo...", expanded=True) as status:
            status.update(label="① Parseando archivo...")
            try:
                data = ei.parse_bitalino_txt(uploaded_file)
                status.update(label=f"✅ {len(data)} muestras, {data.shape[1]} canales")
            except Exception as e:
                st.error(f"Error al parsear: {e}")
                data = None

        if data is not None and len(data) >= 1000:
            wins = ei.sliding_windows(data)
            inf = ei.get_inference()

            results = []
            for i, win in enumerate(wins):
                label, conf, probs = inf.predict(win)
                results.append({
                    "Ventana": i + 1,
                    "Predicción": label,
                    "Confianza (%)": f"{conf:.1f}",
                    "BASAL": f"{probs[0]:.3f}",
                    "NO": f"{probs[1]:.3f}",
                    "SI": f"{probs[2]:.3f}"
                })

```

```

        st.session_state.history.append((label, conf,
"ARCHIVO", time.time()))

        df_res = pd.DataFrame(results)
        st.success(f"✅ {len(wins)} ventanas clasificadas")
        st.dataframe(df_res, use_container_width=True,
hide_index=True)

        # Bar chart of aggregated predictions
        agg = df_res["Predicción"].value_counts()
        chart_df = pd.DataFrame({"Clase": agg.index, "Conteo":
agg.values})
        import plotly.express as px
        fig_agg = px.bar(chart_df, x="Clase", y="Conteo",
                        color="Clase",
                        color_discrete_map={"BASAL": "#636efa",
"NO": "#ef553b", "SI": "#00cc96"},
                        title="Distribución de predicciones por
ventana",
                        text_auto=True, height=300)
        fig_agg.update_layout(showlegend=False)
        st.plotly_chart(fig_agg, use_container_width=True)

        # Raw signal plot
        t = np.arange(min(len(data), 2000)) / 1000
        df_sig = pd.DataFrame({"Tiempo (s)": t})
        for ch in range(4):
            df_sig[f"CH{ch+1}"] = data[:len(t), ch]
        df_plot = df_sig.melt(id_vars="Tiempo (s)",
var_name="Canal", value_name="Amplitud (mV)")
        fig_sig = px.line(df_plot, x="Tiempo (s)", y="Amplitud
(mV)", color="Canal",
                        title="Señal EMG cargada",
                        color_discrete_map={"CH1": "#1f77b4",
"CH2": "#ff7f0e",
"CH3": "#2ca02c",
"CH4": "#d62728"},
                        height=300)
        st.plotly_chart(fig_sig, use_container_width=True)

        st.divider()

```

```

# — Simulación (usa el pipeline real)

st.markdown("### 🖋 Simulación con pipeline real")
col_pipe = st.columns([1, 1])
with col_pipe[0]:
    test_class = st.selectbox("Palabra a simular", ["BASAL", "NO",
"SI"], key="pipe_class")
    with col_pipe[1]:
        st.markdown("***Ejecutar inferencia**")
        run_pipe = st.button("🚀 Clasificar ahora", type="primary",
use_container_width=True)

    if run_pipe:
        inf = ei.get_inference()
        with st.status("Ejecutando pipeline real...", expanded=True) as
status:
            status.update(label="① Generando ventana EMG (1000 samples
× 4 canales)...")
            window = generate_emg_window(st.session_state.persona,
test_class, n_samples=1000)

            status.update(label="② DSP: Welch max-hold FFT 64 → 148
features...")
            features = inf._spectral_analysis(window)

            status.update(label="③ Normalizando (StandardScaler)...")
            features_norm = np.clip((features - inf.mean) / (inf.scale
+ 1e-8), -10, 10)

            status.update(label="④ Inferencia cuantizada int8
(148→120→30→10→3→Softmax)...")
            word, prob, all_probs = inf.predict(window)

            status.update(
                label=f"✅ Clasificación completada — {word} con
{prob:.1f}% de confianza",
                state="complete"
            )

            st.session_state.history.append((word, prob,
st.session_state.persona, time.time()))

```

```

# — Resultado principal


---



res_cols = st.columns([1, 2])

with res_cols[0]:
    st.success(f"### 🎯 {word}")
    st.metric("Confianza", f"{prob:.2f}%", delta=None)

with res_cols[1]:
    st.subheader("Distribución de probabilidades")
    chart_data = pd.DataFrame({
        "Clase": ["BASAL", "NO", "SI"],
        "Probabilidad (%)": [p * 100 for p in all_probs],
    })
    import plotly.express as px
    fig_bar = px.bar(
        chart_data, x="Clase", y="Probabilidad (%)",
        color="Clase",
        color_discrete_map={"BASAL": "#636efa", "NO":
"#ef553b", "SI": "#00cc96"},
        text_auto=".1f",
        height=300,
    )
    fig_bar.update_layout(showlegend=False, yaxis_range=[0,
105])

    st.plotly_chart(fig_bar, use_container_width=True)

# — Features espectrales


---



with st.expander("📊 Ver features espectrales (148-D)",
expanded=False):
    st.write(f"Vector de features normalizado  
(media={features_norm.mean():.3f}, std={features_norm.std():.3f})")

    feat_df = pd.DataFrame({
        "feature": np.arange(148),
        "valor": features_norm
    })
    fig_feat = px.line(
        feat_df, x="feature", y="valor",
        title="148 features espectrales (tras normalización)",
        height=250,
    )

```

```

fig_feat.update_traces(line=dict(width=1))
st.plotly_chart(fig_feat, use_container_width=True)

col_feat = st.columns(2)
with col_feat[0]:
    st.metric("Feature max", f"{features_norm.max():.3f}")
    st.metric("Feature min", f"{features_norm.min():.3f}")
with col_feat[1]:
    st.metric("Norm L2",
f"{np.linalg.norm(features_norm):.3f}")
    st.metric("Ceros", f"{(np.abs(features_norm) <
0.01).sum()} / 148")

# — Últimas predicciones

with st.expander("📖 Historial de predicciones",
expanded=False):
    if st.session_state.history:
        dfh = pd.DataFrame(
            st.session_state.history,
            columns=["Palabra", "Confianza (%)", "Persona",
"Timestamp"]
        )
        dfh["Timestamp"] = pd.to_datetime(dfh["Timestamp"],
unit="s").dt.strftime("%H:%M:%S")
        st.dataframe(dfh.tail(20), use_container_width=True,
hide_index=True)
    else:
        st.info("Aún no hay predicciones.")

else:
    st.info("Selecciona una palabra y presiona **🚀 Clasificar
ahora** para ejecutar el pipeline completo.")

# — Diagrama del modelo

with st.expander("🧠 Arquitectura del modelo neuronal",
expanded=False):
    st.markdown("""
    ...

    Entrada: 148 features (int8)
    |
    [FC1] 148 → 120 | pesos: 148×120 = 17.760 | bias: 120
    """)

```

```

|
[ReLU]
|
[FC2] 120 → 30 | pesos: 120×30 = 3.600 | bias: 30
|
[ReLU]
|
[FC3] 30 → 10 | pesos: 30×10 = 300 | bias: 10
|
[FC4] 10 → 3 | pesos: 10×3 = 30 | bias: 3
|
[Softmax]
|
Salida: {"BASAL", "NO", "SI"} (confianza ≥ 60%)
...

- **Cuantización**: int8 (zero_point=-128, scale=0.00390625)
- **Tensor arena**: 3.539 bytes
- **Compilado con**: Edge Impulse EON Compiler
""")

```

— TAB 4: Acerca del Proyecto

```

with tab_about:
    st.subheader("📄 Acerca del Proyecto EMG + IA")

    col_info = st.columns(2)
    with col_info[0]:
        st.markdown("""
        ### 📄 Ficha técnica

        | Parámetro | Valor |
        |-----|-----|
        | **Dispositivo** | BITalino (r)evolution |
        | **Frecuencia de muestreo** | 1000 Hz |
        | **Canales** | 4 (A1-A4) |
        | **Ventana** | 1000 muestras (1 s) |
        | **Features** | 148 (FFT 64 pts, 4 canales) |
        | **Modelo** | FC 148→120→30→10→3 |
        | **Engine** | TFLite + EON Compiler |
        | **Quantización** | int8 |
        | **Tensor Arena** | 3539 bytes |
        | **Umbral de confianza mínimo** | 60% |
        | **Plataforma** | Edge Impulse (Studio #1042231) |
        """)

```

```

"""

with col_info[1]:
    st.markdown("""
    ### 🖋 Clases

    | Clase | Descripción | Perfil espectral |
    |-----|-----|-----|
    | **BASAL** | Reposo, sin intento de habla | Ruido de fondo
~0.1 mV |
    | **NO** | Intento de articular "NO" | Burst ~30-80 Hz |
    | **SI** | Intento de articular "Sí" | Burst ~80-200 Hz |

    ### 📖 Referencias
    - Edge Impulse Studio:
[#1042231] (https://studio.edgeimpulse.com/studio/1042231)
    - SDK: Edge Impulse Inferencing SDK
    - BITalino: https://bitalino.com
    """)

st.divider()

st.subheader("👥 Personas simuladas")
rows = []
for name, cfg in PERSONAS.items():
    rows.append({
        "Persona": name,
        "SNR (dB)": f"{cfg['snr']:.0f}",
        "Deriva (mV)": f"{cfg['baseline_drift']:.2f}",
        "Amplitud (mV)": f"{cfg['burst_amplitude']:.1f}",
        "Desplaz. freq.": f"{cfg['freq_shift']:+.1f}",
        "Descripción": cfg["desc"]
    })

st.dataframe(pd.DataFrame(rows), use_container_width=True,
hide_index=True)

with st.expander("📖 Glosario técnico", expanded=True):
    st.markdown("""
    - **EMG**: Electromiografía – registro de la actividad
eléctrica muscular
    - **BITalino**: Sistema de adquisición de biosensores low-cost
    - **FFT**: Fast Fourier Transform – convierte señal temporal a
espectro de frecuencias
    """)

```

```
- **EON Compiler**: Compilador de Edge Impulse que genera
código C++ optimizado para embedded
- **StandardScaler**: Normalización que centra en media 0 y
escala a varianza 1
- **Cuantización int8**: Conversión de pesos float32 a enteros
de 8 bits para reducir tamaño y acelerar inferencia
- **Tensor Arena**: Memoria pre-asignada para las activaciones
intermedias del modelo
- **Suprahioideos**: Grupo de 4 músculos (estilohioideo,
digástrico, milohioideo, genihiioideo) que elevan el hioides durante la
deglución y el habla
    """)

st.caption("© Proyecto académico – Ingeniería Biomédica –
Reconocimiento de palabras vía EMG")
```